

WAC and Modern Forms Smart Fans IP API Definition for 3rd Party Integrations

The API allows for retrieving fan information, as well as sending commands to the device.

This file should only be considered accurate for the firmware version it is tagged with in WAC source control.

Copyright 2021 - WAC Lighting

License & Usage

Use of the API's included in this guide are governed by the WAC API License Agreement available for download at: <https://www.waclighting.com/developer>.

Introduction

This document is intended to assist third parties with integrating WAC and Modern Forms smart fans into their control systems. The document is divided into three sections:

- Commissioning and Setup
- Discovery
- Status and Control

WAC and Modern Forms use standard RESTful API's and JSON data formatting.

APIs are tested using CURL (<https://curl.haxx.se/>)

Commissioning and Setup

Fans are delivered from the factory ready to be commissioned to the WAC/Modern Forms cloud through the Modern Forms or WAC app. The fan will start up as a Wi-Fi access point named:

ModernFormsFan_XXXXXX //For Modern Forms fans

WACFan_XXXXXX //For WAC Brand fans

In either case XXXXXX is the last six hexadecimal digits of the Wi-Fi station MAC address for the fan.

NOTE: Generation 3 fan receivers have different MAC Addresses when operating in Station mode versus Access Point mode. All operations using the fan use the station MAC address regardless of operating mode.

Standard procedure is to use the WAC or Modern Forms app, available on the Apple App Store or Google Play to configure the fan to connect to the local Wi-Fi network and register it to the cloud. It is strongly recommended that the fan be setup using the app.

A fan can be commissioned manually using a REST API but it is not supported by WAC/Modern Forms. For information on manual commissioning, please contact WAC technical support.

Discovery

Once commissioned, the fan will connect to the local Wi-Fi network in station mode and will respond to mDNS queries with its service information. WAC and Modern Forms fans can be identified by filtering on type `"easylink"` and protocol `"tcp"`.

Modern Forms will return an `fqdn` value of

`WAC Windermier Fan#XXXXXX._easylink._tcp.local` for Generation 1 and 2 Fans, and `MF_Fan_XXXXXX._easylink._tcp.local` for Generation 3 Fans.

WAC Fans will return an `fqdn` value of `WAC_Fan_XXXXXX._easylink._tcp.local`.

Example response for Generation 3 Fan using Bonjour as a service discovery

```
{
  addresses: ["192.168.1.241"],
  fqdn: "MF_Fan_98E5AC._easylink._tcp.local",
  host: "MF_Fan_98E5AC.local",
  name: "MF_Fan_98E5AC",
  port: 80,
  protocol: "tcp",
  rawTxt: <Buffer(75)>,
  referer: {
    address: "192.168.1.241",
    family: "IPv4",
    port: 5353,
    size: 197,
  },
  subtypes: [],
  txt: {
    firmware rev: "02.99.0011",
    mac: "C8:2B:96:98:E5:AC",
    protocol: "com.modernforms.fan"
  },
  type: "easylink"
}
```

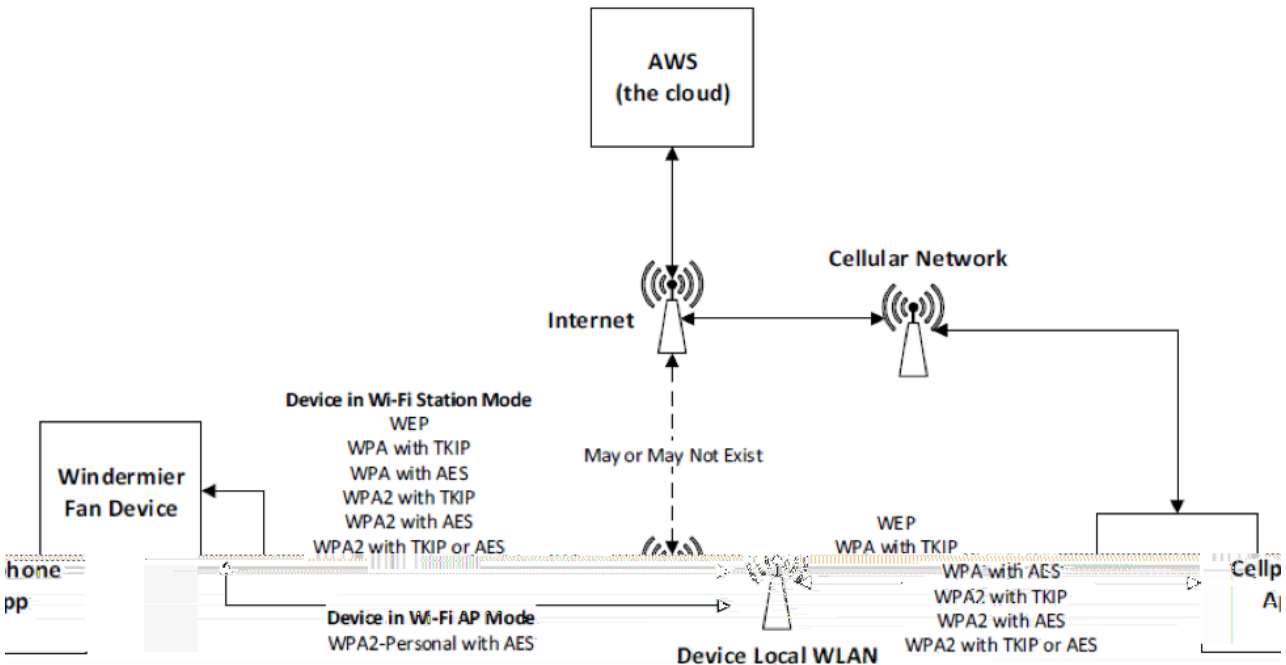


Figure 1: High Level Connections

Status and Control

Read the Fan Status

Retrieve configuration information about the device, such as firmware and hardware versions

Generation 1, 2 Fans

```
ANY /config-read
```

Example request

```
$ curl -X GET 192.168.1.85/config-read
```

Example response

```
{
  "T": "Current Configuration",
  "N": "WAC Windermier Fan(83DEF0)",
  "C": [
    {
      "N": "APPLICATION",
      "C": []
    }
  ],
  "PO": "com.modernforms.fan",
  "HD": "WAC_WINDERMIER_REV_5",
  "FW": "01.03.0021",
  "RF": "wl0: Oct  6 2016 01:32:44 version 5.90.230.15 ",
  "certificateId": "6v6amxh5vbb2qjnkrp2av8i8r1tklsvzwn4ktrr9ds2ljz65ycfq1",
  "Wi-Fi strength": 100
}
```

Property	Description
T	Static string with the value of "Current Configuration"
N	Device Name associated with the Receiver
C	Unused.
PO	Protocol used for mDNS discovery
HD	Hardware revision
FW	Firmware Version
RF	RF library version
certificateId	Identifier associated with the certificates on the device
Wi-Fi strength	WiFi signal strength represented as signal quality percentage

Generation 3 Fans

```
POST /config-read
```

Example request

```
$ curl -X POST 192.168.1.85/config-read
```

Example response

```
{
  "Name": "MF_Fan_98E5AC",
  "Protocol": "com.modernforms.fan",
  "Firmware Rev": "02.00.0003",
  "RF Rev": "v3.2.2",
  "certificateId": "6v6amxh5vbb2qjnkrp2av8i8r1tk1svzwn4ktrr9ds2ljz65ycfq1",
  "Wi-Fi strength": "-48"
}
```

Property	Description
Name	Device Name associated with the Receiver
Protocol	Protocol used for mDNS discovery
Firmware Rev	Firmware Version
RF Rev	RF library version
certificateId	Identifier associated with the certificates on the device
Wi-Fi strength	WiFi signal strength represented as a dB value

Read the Fan Static Shadow Data

Returns the shadow data properties that are not expected to be changed often, such as owner, or never, such as fanType.

Generation 1, 2, 3 Fans

```
POST /mf
```

Example request

```
$ curl -H "Content-Type: application/json" -d '{"queryStaticShadowData":1}'
```

Example request headers

```
{
  "Content-Type": application/json
}
```

Example request body

```
{
  "queryStaticShadowData": 1
}
```

Property	Required/Optional	Data Type	Range
queryStaticShadowData	Required	Integer	[1]

Example response for Generation 1, 2 Fans

```
{
  "clientId": "MF_C8934683DEF0",
  "mac": "C8:93:46:83:DE:F0",
  "lightType": "F4IN-120V-R1-30",
  "fanType": "1803-52",
  "fanMotorType": "DC125X20",
  "productionLotNumber": "",
  "productSku": "",
  "owner": "developer@waclighting.com",
  "federatedIdentity": "us-east-1:ef3pcxfz-2fngy-fn1cth-0kw0q0-ziitsbiz2w",
  "deviceName": "Living Room",
  "firmwareVersion": "01.03.0021",
  "mainMcuFirmwareVersion": "01.03.3008",
  "firmwareUrl": ""
}
```

Property	Description	Data Type	Range
clientId	Unique identifier specific to a single device	String	MF_XXXXXXXXXXXX or WAC_XXXXXXXXXXXX
mac	The Media Access Control address of the fan, used to uniquely identify this device on the network. The x's are the ASCII hex representation of the device's 6 MAC address bytes.	String	xx:xx:xx:xx:xx:xx
	The type of light assembly installed		

<code>lightType</code>	in the device, if applicable	String	printable ASCII
<code>fanType</code>	Combination string of fan model number and blade length	String	printable ASCII
<code>fanMotorType</code>	Identifying string for the motor type in the fan	String	printable ASCII
<code>productionLotNumber</code>	The production lot that the device was manufactured in	String	printable ASCII
<code>productSku</code>	The SKU # of the device	String	printable ASCII
<code>owner</code>	The email address associated with the account used to pair the device	String	printable ASCII
<code>federatedIdentity</code>	Unique identifier from AWS specific to a single user	String	printable ASCII
<code>deviceName</code>	The name given to the device when paired or edited in the app	String	printable ASCII
<code>firmwareVersion</code>	The current firmware version the device is using	String	printable ASCII
	The current version of firmware that is the application-specific MCU that		

<code>mainMcuFirmwareVersion</code>	electrically controls the fan. This is not needed by anything right now but may be used in the future for things like RMA units. The d's are digits 0-9, where the first section represents the major version, second is the minor version, and last section is the build version	String	dd.dd.dddd
<code>firmwareUrl</code>	URL that tells the device where to download the latest version of firmware	String	printable ASCII

Example response for Generation 3 Fans

```
{
  "clientId": "MF_C82B9698E5AC",
  "mac": "C8:2B:96:98:E5:AC",
  "lightType": "",
  "fanType": "2003-52",
  "fanMotorType": "DC125X12",
  "brand": 0,
  "dateCode": "",
  "owner": "developer@waclighting.com",
  "federatedIdentity": "us-east-1:ef3pcxfz-2fngy-fn1cth-0kw0q0-ziitsbiz2w",
  "deviceName": "Living Room",
  "firmwareVersion": "02.00.0003",
  "mainMcuFirmwareVersion": "02.01.0000",
  "firmwareUrl": ""
}
```

Property	Description	Data Type	Range
----------	-------------	-----------	-------

<code>clientId</code>	Unique identifier specific to a single device	String	MF_XXXXXXXXXXXX or WAC_XXXXXXXXXXXX
<code>mac</code>	The Media Access Control address of the fan, used to uniquely identify this device on the network. The x's are the ASCII hex representation of the device's 6 MAC address bytes.	String	xx:xx:xx:xx:xx:xx
<code>lightType</code>	The type of light assembly installed in the device, if applicable	String	printable ASCII
<code>fanType</code>	Combination string of fan model number and blade length	String	printable ASCII
<code>fanMotorType</code>	Identifying string for the motor type in the fan	String	printable ASCII
<code>brand</code>	Integer used to indicate a device's brand	Integer	[0 - 2]
<code>dateCode</code>	Date that certificates were loaded onto the device	String	printable ASCII
<code>owner</code>	The email address associated with the account used to	String	printable ASCII

	pair the device		
<code>federatedIdentity</code>	Unique identifier from AWS specific to a single user	String	printable ASCII
<code>deviceName</code>	The name given to the device when paired or edited in the app	String	printable ASCII
<code>firmwareVersion</code>	The current firmware version the device is using	String	printable ASCII
<code>mainMcuFirmwareVersion</code>	The current version of firmware that is the application-specific MCU that electrically controls the fan. This is not needed by anything right now but may be used in the future for things like RMA units. The d's are digits 0-9, where the first section represents the major version, second is the minor version, and last section is the build version	String	dd.dd.dddd
<code>firmwareUrl</code>	URL that tells the device where to download the	String	printable ASCII

	latest version of firmware		
--	-------------------------------	--	--



Read the Fan Dynamic Shadow Data

Returns the shadow data properties that change often during normal operation, such as the light brightness or fan speed value.

Generation 1, 2, 3 Fans

```
POST /mf
```

Example request

```
$ curl -H "Content-Type: application/json" -d '{"queryDynamicShadowData":1}'
```

Example request headers

```
{
  "Content-Type": application/json
}
```

Example request body

```
{
  "queryDynamicShadowData": 1
}
```

Property	Required/Optional	Data Type	Range
queryDynamicShadowData	Required	Integer	[1]

Example response for Generation 1, 2 Fans

```

{
  "clientId": "MF_C8934683DEF0",
  "lightOn": false,
  "fanOn": false,
  "lightBrightness": 25,
  "fanSpeed": 1,
  "fanDirection": "forward",
  "rfPairModeActive": false,
  "resetRfPairList": false,
  "factoryReset": false,
  "awayModeEnabled": false,
  "fanSleepTimer": 0,
  "lightSleepTimer": 0,
  "decommission": false,
  "schedule": "AAAAAPwDAGSgBQAAnAkAZEALAAA8DwBk4BAAANwUAGSAFgAAfBoAZCacAA
  "adaptiveLearning": false
}

```

In the following table, the use of the word "device" refers to the entire product as a whole, while "fan" and "light" refer to the individual components themselves.

Property	Description	Data Type	Range
clientId	Unique identifier specific to a single device	String	MF_XXXXXXXXXXXX or WAC_XXXXXXXXXXXX
lightOn	Indicates if the light is On or Off	Boolean	[true, false]
lightBrightness	Controls the brightness of the light	Integer	[1-100]
fanOn	Indicates if fan is On or Off	Boolean	[true, false]
fanSpeed	Controls the speed of the fan	Integer	[1-6]
fanDirection	Indicates if fan is in Summer Mode (forward) or Winter	String	["forward", "reverse"]

	Mode (<code>reverse</code>)		
<code>rfPairModeActive</code>	Indicates if the device is in Pairing Mode. Set to <code>true</code> to pair devices such as remotes or wall controls	Boolean	[true, false]
<code>resetRfPairList</code>	Boolean value indicating if device should clear all paired devices. Setting to <code>true</code> would clear all paired devices	Boolean	[true, false]
<code>factoryReset</code>	Boolean value indicating if device should be factory reset. Setting to <code>true</code> would initiate factory reset process. This type of reset erases all home AP credentials (SSID and password), puts the device back into Wi-Fi AP mode, decommisions the device from the cloud, and clears all previously paired devices	Boolean	[true, false]
<code>awayModeEnabled</code>	Boolean value indicating if device should be operate in Away Mode. Away Mode will control the device to simulate that a user is present	Boolean	[true, false]
<code>fanSleepTimer</code>	Epoch timestamp at which the fan should turn off	Integer	[0-2147483647]
<code>lightSleepTimer</code>	Epoch timestamp at which the light should turn	Integer	[0-2147483647]

	off		
decommission			

```

{
  "clientId": "MF_C82B9698E5AC",
  "lightOn": true,
  "fanOn": true,
  "lightBrightness": 100,
  "fanSpeed": 1,
  "fanDirection": "forward",
  "wind": false,
  "windSpeed": 2,
  "rfPairModeActive": false,
  "resetRfPairList": false,
  "factoryReset": false,
  "awayModeEnabled": false,
  "fanTimer": 0,
  "lightTimer": 0,
  "decommission": false,
  "schedule": "AAAAAPwDAGSgBQAAnAkAZEALAAA8DwBk4BAAANwUAGSAFgAAfBoAZCacAA",
  "adaptiveLearning": false,
  "userData": "cloud"
}

```

In the following table, the use of the word "device" refers to the entire product as a whole, while "fan" and "light" refer to the individual components themselves.

Property	Description	Data Type	Range
clientId	Unique identifier specific to a single device	String	MF_XXXXXXXXXXXX or WAC_XXXXXXXXXXXX
lightOn	Indicates if the light is On or Off	Boolean	[true, false]
lightBrightness	Controls the brightness of the light	Integer	[1-100]
fanOn	Indicates if fan is On or Off	Boolean	[true, false]
fanSpeed	Controls the speed of the fan	Integer	[1-6]
	Indicates if fan is in		

<code>fanDirection</code>	Summer Mode (<code>forward</code>) or Winter Mode (<code>reverse</code>)	String	["forward", "reverse"]
<code>wind</code>	Boolean value indicating if device is in operating in Wind Mode (<code>true</code>) or not (<code>false</code>). Wind Mode mimics a natural breeze by setting a base speed, and then increasing the fan speed at randomly timed intervals to simulate a gust of wind, then returning to the base speed	Boolean	[true, false]
<code>windSpeed</code>	Integer value ranging from 1-3 indicating the strength of the base and gust fan speeds	Integer	[1-3]
<code>rfPairModeActive</code>	Indicates if the device is in Pairing Mode. Set to <code>true</code> to pair devices such as remotes or wall controls	Boolean	[true, false]
<code>resetRfPairList</code>	Boolean value indicating if device should clear all paired devices. Setting to <code>true</code> would clear all paired devices	Boolean	[true, false]
	Boolean value indicating if device should be factory reset. Setting to <code>true</code> would initiate factory reset process. This type of reset erases all home AP		

<code>factoryReset</code>	credentials (SSID and password), puts the device back into Wi-Fi AP mode, decommissions the device from the cloud, and clears all previously paired devices	Boolean	[true, false]
<code>awayModeEnabled</code>	Boolean value indicating if device should be operate in Away Mode. Away Mode will control the device to simulate that a user is present	Boolean	[true, false]
<code>fanTimer</code>	Integer value indicting how many second until the fan should turn off	Integer	[0-2147483647]
<code>lightTimer</code>	Integer value indicting how many second until the light should turn off	Integer	[0-2147483647]
<code>decommission</code>	Boolean value indicating if device should be decommissioned. Setting to <code>true</code> would initiate the decommission process. The device will decommission itself from the cloud, if it has access, then set Wi-Fi credentials back to factory default settings and enter AP mode.	Boolean	[true, false]
<code>schedule</code>	Base64 Encoded string value representing scheduled events to control fan power and speed, and light power	String	All Base64 Bytes

	and brightness		
<code>adaptiveLearning</code>	Boolean value indicating if device should be operate in Adaptive Learning Mode. Adaptive Learning Mode will study a user's behavior and control the device to mimic their habits	Boolean	[true, false]
<code>userData</code>	Indicates the last method used to control the device	String	["none", "local_wifi", "cloud", "ble", "sleep_timer", "away_mode", "schedule"]

Control the Fan

Send commands to the device, such as turning the fan on or setting the light brightness to 100.

Generation 1, 2, 3 Fans

```
POST /mf
```

Example request

```
$ curl -H "Content-Type: application/json" -d '{"lightOn": true, "lightBrig
```

Example request headers

```
{
  "Content-Type": application/json
}
```

Example request body

```
{
  "lightOn": true,
  "lightBrightness": 100
}
```

Property	Required/Optional	Data Type	Range
lightOn	Optional	Boolean	[true, false]
lightBrightness	Optional	Integer	[1-100]
fanOn	Optional	Boolean	[true, false]
fanSpeed	Optional	Integer	[1-6]
fanDirection	Optional	String	["forward", reverse]
wind	Optional (<i>Gen 3+ Only</i>)	Boolean	[true, false]
windSpeed	Optional (<i>Gen 3+ Only</i>)	Integer	[1-3]

Example response

Reponse will be the dynamic shadow data, with the updated values

```
{
  "clientId": "MF_C82B9698E5AC",
  "lightOn": true,
  "fanOn": true,
  "lightBrightness": 100,
  "fanSpeed": 1,
  "fanDirection": "forward",
  "wind": false,
  "windSpeed": 2,
  "rfPairModeActive": false,
  "resetRfPairList": false,
  "factoryReset": false,
  "awayModeEnabled": false,
  "fanTimer": 0,
  "lightTimer": 0,
  "decommission": false,
  "schedule": "",
  "adaptiveLearning": false,
  "userData": "local_wifi"
}
```